

MySQL Commands Complete Reference Guide

Syntax, Descriptions & Worked Examples for Teaching

Compiled July 2026

This guide is organized by command category — Data Definition, Data Manipulation, Data Query, Joins, Transactions, User & Privilege Management, Indexes/Views, and built-in Functions. Each entry includes a plain-language description, the general syntax, and a runnable example, making it suitable for classroom hand-outs, lab exercises, or self-study.

Contents

1. Introduction to MySQL & Database Basics
2. Data Definition Language (DDL)
3. Constraints
4. Data Manipulation Language (DML)
5. Data Query Language (DQL) — SELECT & Clauses
6. Joins
7. Subqueries & Set Operations
8. Aggregate & Built-in Functions
9. Indexes & Views
10. Transaction Control Language (TCL)
11. Data Control Language (DCL) — Users & Privileges
12. Stored Procedures, Functions & Triggers

1. Introduction to MySQL

MySQL is an open-source relational database management system (RDBMS) that uses Structured Query Language (SQL) to define, manipulate, and query data stored in tables. Commands are broadly grouped into five categories:

Category	Purpose	Example Commands
DDL	Define/modify database structure	CREATE, ALTER, DROP, TRUNCATE
DML	Modify data inside tables	INSERT, UPDATE, DELETE
DQL	Query/retrieve data	SELECT
DCL	Control access & permissions	GRANT, REVOKE
TCL	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

Connecting to the MySQL Server

Opens the MySQL command-line client and prompts for the account password. Once connected, the prompt changes to **mysql>** and SQL statements can be entered.

SYNTAX

```
mysql -u <username> -p
```

EXAMPLE

```
mysql -u root -p
```

SHOW DATABASES

Lists every database currently hosted on the MySQL server that the connected user is permitted to see.

SYNTAX

```
SHOW DATABASES;
```

EXAMPLE

```
SHOW DATABASES;
```

USE

Selects a database as the current working database. Every unqualified table name after this statement refers to a table inside the chosen database.

SYNTAX

```
USE database_name;
```

EXAMPLE

```
USE school_db;
```

2. Data Definition Language (DDL)

DDL statements create, modify, and remove database objects such as databases, tables, and schemas. DDL changes are auto-committed in MySQL — they take effect immediately.

CREATE DATABASE

Creates a new, empty database on the server. The optional IF NOT EXISTS clause prevents an error if a database with that name already exists.

SYNTAX

```
CREATE DATABASE [IF NOT EXISTS] database_name;
```

EXAMPLE

```
CREATE DATABASE IF NOT EXISTS school_db;
```

DROP DATABASE

Permanently deletes a database and every table, view, and row inside it. This action cannot be undone, so it should be used with care.

SYNTAX

```
DROP DATABASE [IF EXISTS] database_name;
```

EXAMPLE

```
DROP DATABASE IF EXISTS old_test_db;
```

Note: Always double-check the database name before running DROP in a live environment.

CREATE TABLE

Creates a new table with the specified columns, data types, and constraints (such as PRIMARY KEY or NOT NULL).

SYNTAX

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
);
```

EXAMPLE

```
CREATE TABLE students (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name  VARCHAR(50) NOT NULL,  
    email      VARCHAR(100) UNIQUE,  
    enrolled_on DATE DEFAULT (CURRENT_DATE)  
);
```

ALTER TABLE ... ADD COLUMN

Adds a new column to an existing table without affecting the rows already stored there (new column values default to NULL unless a DEFAULT is specified).

SYNTAX

```
ALTER TABLE table_name ADD COLUMN column_name datatype;
```

EXAMPLE

```
ALTER TABLE students ADD COLUMN phone VARCHAR(15);
```

ALTER TABLE ... MODIFY COLUMN

Changes the data type or definition of an existing column, for example widening a VARCHAR or changing a column to allow NULL values.

SYNTAX

```
ALTER TABLE table_name MODIFY COLUMN column_name new_datatype;
```

EXAMPLE

```
ALTER TABLE students MODIFY COLUMN phone VARCHAR(20);
```

ALTER TABLE ... DROP COLUMN

Removes a column and all of its data from a table permanently.

SYNTAX

```
ALTER TABLE table_name DROP COLUMN column_name;
```

EXAMPLE

```
ALTER TABLE students DROP COLUMN phone;
```

ALTER TABLE ... RENAME

Renames an existing table without touching its data or structure.

SYNTAX

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

EXAMPLE

```
ALTER TABLE students RENAME TO learners;
```

TRUNCATE TABLE

Quickly removes all rows from a table and resets any AUTO_INCREMENT counter. It is faster than DELETE because it does not log individual row deletions, but it cannot be rolled back and skips row-level triggers.

SYNTAX

```
TRUNCATE TABLE table_name;
```

EXAMPLE

```
TRUNCATE TABLE students;
```

DROP TABLE

Deletes a table's structure and all of its data permanently.

SYNTAX

```
DROP TABLE [IF EXISTS] table_name;
```

EXAMPLE

```
DROP TABLE IF EXISTS temp_scores;
```

RENAME TABLE

An alternative way to rename one or more tables in a single statement.

SYNTAX

```
RENAME TABLE old_name TO new_name;
```

EXAMPLE

```
RENAME TABLE learners TO students, old_grades TO grades;
```

3. Constraints

Constraints enforce rules on the data stored in a table, protecting its integrity. They are usually defined when a table is created, but can also be added afterward with ALTER TABLE.

PRIMARY KEY

Uniquely identifies each row in a table. A table can have only one primary key, its values must be unique, and it cannot contain NULL.

SYNTAX

```
column_name datatype PRIMARY KEY
-- or --
PRIMARY KEY (column_name)
```

EXAMPLE

```
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  title VARCHAR(100)
);
```

FOREIGN KEY

Links a column to the primary key of another table, enforcing that any value entered must already exist in the referenced table (referential integrity).

SYNTAX

```
FOREIGN KEY (column_name) REFERENCES other_table(other_column)
```

EXAMPLE

```
CREATE TABLE enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

NOT NULL

Requires that a column always contain a value; INSERT or UPDATE statements that omit it will fail.

SYNTAX

```
column_name datatype NOT NULL
```

EXAMPLE

```
email VARCHAR(100) NOT NULL
```

UNIQUE

Ensures that every value in a column is different across all rows, while still allowing one NULL.

SYNTAX

```
column_name datatype UNIQUE
```

EXAMPLE

```
email VARCHAR(100) UNIQUE
```

DEFAULT

Supplies an automatic value for a column when no value is given in an INSERT statement.

SYNTAX

```
column_name datatype DEFAULT value
```

EXAMPLE

```
status VARCHAR(20) DEFAULT 'active'
```

CHECK

Restricts the values allowed in a column to those that satisfy a logical condition (MySQL 8.0.16+).

SYNTAX

```
CHECK (condition)
```

EXAMPLE

```
age INT CHECK (age >= 0)
```

4. Data Manipulation Language (DML)

DML statements insert, change, and remove the rows stored inside tables.

INSERT INTO

Adds one new row to a table with the specified column values. Columns left out receive their default value or NULL.

SYNTAX

```
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

EXAMPLE

```
INSERT INTO students (first_name, last_name, email)
VALUES ('Ana', 'Reyes', 'ana.reyes@example.com');
```

INSERT ... Multiple Rows

Inserts several rows in a single statement, which is more efficient than issuing one INSERT per row.

SYNTAX

```
INSERT INTO table_name (col1, col2) VALUES (v1a, v1b), (v2a, v2b), ...;
```

EXAMPLE

```
INSERT INTO students (first_name, last_name) VALUES
('Liam', 'Chen'),
('Priya', 'Nair'),
('Omar', 'Haddad');
```

UPDATE

Modifies existing rows that match the WHERE condition. Omitting WHERE updates every row in the table, so it should always be checked carefully.

SYNTAX

```
UPDATE table_name SET col1 = val1, col2 = val2 WHERE condition;
```

EXAMPLE

```
UPDATE students
SET email = 'ana.r@example.com'
WHERE student_id = 1;
```

DELETE

Removes rows that match the WHERE condition. Like UPDATE, leaving out WHERE deletes every row in the table.

SYNTAX

```
DELETE FROM table_name WHERE condition;
```

EXAMPLE

```
DELETE FROM students WHERE student_id = 5;
```

REPLACE INTO

Inserts a row, but if a row with the same primary key or unique key already exists it is deleted first and replaced — effectively an upsert.

SYNTAX

```
REPLACE INTO table_name (col1, col2) VALUES (val1, val2);
```

EXAMPLE

```
REPLACE INTO students (student_id, first_name, last_name)
VALUES (1, 'Ana', 'Reyes-Lopez');
```

5. Data Query Language (DQL)

The SELECT statement retrieves data from one or more tables. Its clauses can be combined to filter, sort, group, and limit results.

SELECT (basic)

Retrieves the specified columns from a table. Use SELECT * to return every column.

SYNTAX

```
SELECT column1, column2 FROM table_name;
```

EXAMPLE

```
SELECT first_name, last_name FROM students;
```

WHERE

Filters rows so that only those satisfying the condition are returned. Supports comparison operators (=, <>, >, <), logical operators (AND, OR, NOT), and pattern/range operators (LIKE, BETWEEN, IN).

SYNTAX

```
SELECT columns FROM table_name WHERE condition;
```

EXAMPLE

```
SELECT * FROM students WHERE last_name = 'Reyes';
```

ORDER BY

Sorts the result set by one or more columns, ascending by default (ASC) or descending (DESC).

SYNTAX

```
SELECT columns FROM table_name ORDER BY column [ASC|DESC];
```

EXAMPLE

```
SELECT first_name, enrolled_on FROM students  
ORDER BY enrolled_on DESC;
```

GROUP BY

Groups rows that share the same value in one or more columns so an aggregate function (COUNT, SUM, AVG, etc.) can be applied to each group.

SYNTAX

```
SELECT column, AGG_FUNC(column2) FROM table_name GROUP BY column;
```

EXAMPLE

```
SELECT course_id, COUNT(*) AS total_students  
FROM enrollments  
GROUP BY course_id;
```

HAVING

Filters groups created by GROUP BY, similar to WHERE but applied after aggregation.

SYNTAX

```
SELECT columns FROM table_name GROUP BY column HAVING condition;
```

EXAMPLE

```
SELECT course_id, COUNT(*) AS total_students  
FROM enrollments  
GROUP BY course_id
```

```
HAVING COUNT(*) > 10;
```

LIMIT / OFFSET

Restricts the number of rows returned, useful for pagination or previewing data.

SYNTAX

```
SELECT columns FROM table_name LIMIT count OFFSET skip;
```

EXAMPLE

```
SELECT * FROM students ORDER BY student_id LIMIT 10 OFFSET 20;
```

DISTINCT

Removes duplicate values from the result set, returning only unique rows.

SYNTAX

```
SELECT DISTINCT column FROM table_name;
```

EXAMPLE

```
SELECT DISTINCT last_name FROM students;
```

LIKE

Matches text against a pattern using wildcards: % for any sequence of characters, and _ for a single character.

SYNTAX

```
SELECT columns FROM table_name WHERE column LIKE pattern;
```

EXAMPLE

```
SELECT * FROM students WHERE email LIKE '%@gmail.com';
```

BETWEEN

Selects rows whose column value falls within an inclusive range.

SYNTAX

```
SELECT columns FROM table_name WHERE column BETWEEN val1 AND val2;
```

EXAMPLE

```
SELECT * FROM students WHERE enrolled_on BETWEEN '2025-01-01' AND '2025-06-30';
```

IN

Checks whether a column's value matches any value in a given list — a shorthand for multiple OR conditions.

SYNTAX

```
SELECT columns FROM table_name WHERE column IN (val1, val2, ...);
```

EXAMPLE

```
SELECT * FROM students WHERE last_name IN ('Reyes', 'Chen', 'Nair');
```

6. Joins

Joins combine rows from two or more tables based on a related column, typically a foreign key.

INNER JOIN

Returns only the rows that have matching values in both tables.

SYNTAX

```
SELECT columns FROM tableA
INNER JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT s.first_name, c.title
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

LEFT JOIN

Returns all rows from the left table, plus matching rows from the right table; unmatched right-side columns appear as NULL.

SYNTAX

```
SELECT columns FROM tableA
LEFT JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT s.first_name, e.course_id
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id;
```

RIGHT JOIN

Returns all rows from the right table, plus matching rows from the left table; unmatched left-side columns appear as NULL.

SYNTAX

```
SELECT columns FROM tableA
RIGHT JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT s.first_name, e.course_id
FROM students s
RIGHT JOIN enrollments e ON s.student_id = e.student_id;
```

CROSS JOIN

Returns the Cartesian product of both tables — every row of the first table paired with every row of the second.

SYNTAX

```
SELECT columns FROM tableA CROSS JOIN tableB;
```

EXAMPLE

```
SELECT s.first_name, c.title
FROM students s
CROSS JOIN courses c;
```

SELF JOIN

Joins a table to itself, useful for comparing rows within the same table such as employees and their managers.

SYNTAX

```
SELECT a.col, b.col FROM table_name a
JOIN table_name b ON a.related_col = b.id;
```

EXAMPLE

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id;
```

7. Subqueries & Set Operations

A subquery is a `SELECT` statement nested inside another query. Set operations combine the results of two separate `SELECT` statements.

Subquery in WHERE

Uses the result of an inner query to filter the outer query's rows.

SYNTAX

```
SELECT columns FROM table_name
WHERE column OPERATOR (SELECT column FROM other_table WHERE condition);
```

EXAMPLE

```
SELECT first_name FROM students
WHERE student_id IN (
    SELECT student_id FROM enrollments WHERE course_id = 3
);
```

Subquery in FROM (derived table)

Treats the result of an inner query as a temporary table that the outer query can select from, filter, or join.

SYNTAX

```
SELECT columns FROM (SELECT ... FROM table_name) AS alias;
```

EXAMPLE

```
SELECT course_id, total
FROM (
    SELECT course_id, COUNT(*) AS total
    FROM enrollments GROUP BY course_id
) AS counts
WHERE total > 5;
```

UNION

Combines the results of two queries into one result set and removes duplicate rows. Both queries must return the same number of columns with compatible types.

SYNTAX

```
SELECT columns FROM tableA
UNION
SELECT columns FROM tableB;
```

EXAMPLE

```
SELECT email FROM students
UNION
SELECT email FROM instructors;
```

UNION ALL

Combines the results of two queries like `UNION`, but keeps duplicate rows and is faster because it skips the duplicate check.

SYNTAX

```
SELECT columns FROM tableA
UNION ALL
SELECT columns FROM tableB;
```

EXAMPLE

```
SELECT email FROM students  
UNION ALL  
SELECT email FROM instructors;
```

8. Aggregate & Built-in Functions

MySQL provides built-in functions for summarizing data, manipulating text, and working with dates.

COUNT()

Counts the number of rows, or the number of non-NULL values in a specific column.

SYNTAX

```
SELECT COUNT(column) FROM table_name;
```

EXAMPLE

```
SELECT COUNT(*) FROM students;
```

SUM()

Adds together all the numeric values in a column.

SYNTAX

```
SELECT SUM(column) FROM table_name;
```

EXAMPLE

```
SELECT SUM(amount) FROM payments;
```

AVG()

Calculates the average value of a numeric column.

SYNTAX

```
SELECT AVG(column) FROM table_name;
```

EXAMPLE

```
SELECT AVG(grade) FROM exam_results;
```

MIN() / MAX()

Returns the smallest and largest values found in a column.

SYNTAX

```
SELECT MIN(column), MAX(column) FROM table_name;
```

EXAMPLE

```
SELECT MIN(grade), MAX(grade) FROM exam_results;
```

CONCAT()

Joins two or more strings (or column values) into a single string.

SYNTAX

```
SELECT CONCAT(str1, str2, ...) FROM table_name;
```

EXAMPLE

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM students;
```

UPPER() / LOWER()

Converts text to all uppercase or all lowercase letters.

SYNTAX

```
SELECT UPPER(column), LOWER(column) FROM table_name;
```

EXAMPLE

```
SELECT UPPER(last_name) FROM students;
```

SUBSTRING()

Extracts a portion of a string, starting at a given position for a given length.

SYNTAX

```
SELECT SUBSTRING(column, start, length) FROM table_name;
```

EXAMPLE

```
SELECT SUBSTRING(email, 1, 5) FROM students;
```

NOW() / CURDATE()

NOW() returns the current date and time; CURDATE() returns only the current date.

SYNTAX

```
SELECT NOW();  
SELECT CURDATE();
```

EXAMPLE

```
SELECT NOW() AS server_time, CURDATE() AS today;
```

DATEDIFF()

Returns the number of days between two dates.

SYNTAX

```
SELECT DATEDIFF(date1, date2);
```

EXAMPLE

```
SELECT DATEDIFF(CURDATE(), enrolled_on) AS days_enrolled FROM students;
```

ROUND()

Rounds a numeric value to the specified number of decimal places.

SYNTAX

```
SELECT ROUND(number, decimals);
```

EXAMPLE

```
SELECT ROUND(AVG(grade), 2) FROM exam_results;
```

IFNULL() / COALESCE()

IFNULL replaces a NULL value with a fallback value. COALESCE returns the first non-NULL value from a list of expressions.

SYNTAX

```
SELECT IFNULL(column, replacement);  
SELECT COALESCE(col1, col2, ...);
```

EXAMPLE

```
SELECT IFNULL(phone, 'Not provided') FROM students;
```

9. Indexes & Views

Indexes speed up data retrieval, and views provide a saved, reusable SELECT query that behaves like a virtual table.

CREATE INDEX

Builds an index on one or more columns so lookups and WHERE/ORDER BY on that column run faster, at the cost of extra storage and slightly slower writes.

SYNTAX

```
CREATE INDEX index_name ON table_name (column);
```

EXAMPLE

```
CREATE INDEX idx_lastname ON students (last_name);
```

DROP INDEX

Removes an existing index from a table.

SYNTAX

```
DROP INDEX index_name ON table_name;
```

EXAMPLE

```
DROP INDEX idx_lastname ON students;
```

CREATE VIEW

Saves a SELECT statement under a name so it can be reused like a table, without duplicating the underlying data.

SYNTAX

```
CREATE VIEW view_name AS SELECT ... FROM table_name WHERE condition;
```

EXAMPLE

```
CREATE VIEW honor_roll AS
SELECT first_name, last_name FROM students s
JOIN exam_results r ON s.student_id = r.student_id
WHERE r.grade >= 90;
```

DROP VIEW

Deletes a view definition. The underlying tables and their data are not affected.

SYNTAX

```
DROP VIEW [IF EXISTS] view_name;
```

EXAMPLE

```
DROP VIEW IF EXISTS honor_roll;
```

10. Transaction Control Language (TCL)

Transactions group multiple statements so they succeed or fail together, preserving data consistency. TCL commands require a storage engine that supports transactions, such as InnoDB.

START TRANSACTION

Begins a new transaction. Statements that follow are not permanently saved until COMMIT is issued.

SYNTAX

```
START TRANSACTION;
```

EXAMPLE

```
START TRANSACTION;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
```

COMMIT

Permanently saves all changes made during the current transaction.

SYNTAX

```
COMMIT;
```

EXAMPLE

```
COMMIT;
```

ROLLBACK

Undoes all changes made during the current transaction, returning the data to its state before START TRANSACTION.

SYNTAX

```
ROLLBACK;
```

EXAMPLE

```
ROLLBACK;
```

SAVEPOINT

Marks a point within a transaction that a later ROLLBACK can return to, without undoing the entire transaction.

SYNTAX

```
SAVEPOINT savepoint_name;
```

EXAMPLE

```
SAVEPOINT before_update;  
UPDATE accounts SET balance = balance - 50 WHERE id = 1;  
ROLLBACK TO before_update;
```

11. Data Control Language (DCL)

DCL statements manage user accounts and control what actions each account is allowed to perform.

CREATE USER

Creates a new MySQL account with a login name, allowed host, and password.

SYNTAX

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

EXAMPLE

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'StrongPass123!';
```

GRANT

Gives a user account specific permissions, such as the ability to SELECT, INSERT, UPDATE, or DELETE on a database or table.

SYNTAX

```
GRANT privilege_list ON database.table TO 'username'@'host';
```

EXAMPLE

```
GRANT SELECT, INSERT, UPDATE ON school_db.* TO 'app_user'@'localhost';
```

REVOKE

Removes previously granted permissions from a user account.

SYNTAX

```
REVOKE privilege_list ON database.table FROM 'username'@'host';
```

EXAMPLE

```
REVOKE INSERT, UPDATE ON school_db.* FROM 'app_user'@'localhost';
```

DROP USER

Deletes a user account and all of its granted privileges.

SYNTAX

```
DROP USER 'username'@'host';
```

EXAMPLE

```
DROP USER 'app_user'@'localhost';
```

FLUSH PRIVILEGES

Reloads the grant tables so that privilege changes made directly to system tables take effect immediately (usually unnecessary after GRANT/REVOKE, which apply automatically).

SYNTAX

```
FLUSH PRIVILEGES;
```

EXAMPLE

```
FLUSH PRIVILEGES;
```

SHOW GRANTS

Displays the privileges currently assigned to a user account.

SYNTAX

```
SHOW GRANTS FOR 'username'@'host';
```

EXAMPLE

```
SHOW GRANTS FOR 'app_user'@'localhost';
```

12. Stored Procedures, Functions & Triggers

Stored programs let repeated logic live inside the database itself, so it can be reused and executed on the server.

CREATE PROCEDURE

Defines a stored procedure — a named block of SQL statements that can be executed with CALL. Parameters allow values to be passed in and out.

SYNTAX

```
DELIMITER //
CREATE PROCEDURE procedure_name (parameters)
BEGIN
    -- statements
END //
DELIMITER ;
```

EXAMPLE

```
DELIMITER //
CREATE PROCEDURE GetStudentCount(IN course INT, OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM enrollments WHERE course_id = course;
END //
DELIMITER ;
```

CALL

Executes a stored procedure that has already been created.

SYNTAX

```
CALL procedure_name(arguments);
```

EXAMPLE

```
CALL GetStudentCount(3, @result);
SELECT @result;
```

CREATE FUNCTION

Defines a reusable, named function that returns a single value and can be called directly inside SQL expressions.

SYNTAX

```
DELIMITER //
CREATE FUNCTION function_name (parameters) RETURNS datatype
BEGIN
    -- statements
    RETURN value;
END //
DELIMITER ;
```

EXAMPLE

```
DELIMITER //
CREATE FUNCTION FullName(fn VARCHAR(50), ln VARCHAR(50))
RETURNS VARCHAR(101) DETERMINISTIC
BEGIN
    RETURN CONCAT(fn, ' ', ln);
END //
DELIMITER ;
```

CREATE TRIGGER

Defines an action that runs automatically whenever a specified event (INSERT, UPDATE, or DELETE) happens on a table.

SYNTAX

```
CREATE TRIGGER trigger_name
{BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON table_name
FOR EACH ROW
BEGIN
  -- statements
END;
```

EXAMPLE

```
CREATE TRIGGER before_student_insert
BEFORE INSERT ON students
FOR EACH ROW
SET NEW.enrolled_on = IFNULL(NEW.enrolled_on, CURDATE());
```

DROP PROCEDURE / FUNCTION / TRIGGER

Removes a stored procedure, function, or trigger definition from the database.

SYNTAX

```
DROP PROCEDURE IF EXISTS name;
DROP FUNCTION IF EXISTS name;
DROP TRIGGER IF EXISTS name;
```

EXAMPLE

```
DROP PROCEDURE IF EXISTS GetStudentCount;
```

End of reference guide. This document is intended as a teaching aid — encourage students to run each example against a practice database to see the results firsthand.